MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Better Metrics for LLM Inference Serving Systems

Arnav Reddy, Nishant Dash, Muzhe Wu, Yuxuan Liu

Existing Metrics

- Throughput
  - #Tokens / Time (s)
- Latency
  - Time Per Output Token (TPOT)
  - Time To First Token (TTFT)
  - Time To Last Token (TTLT)
  - Time Between Tokens (TBT)

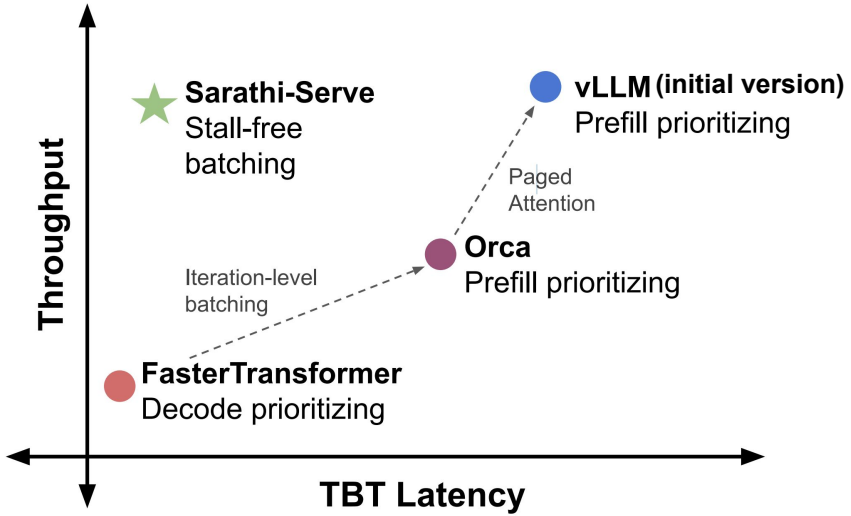


Image from Sarathi Serve

Are these metrics (good) enough for evaluating/informing the design of LLM Inference Serving System?

arXiv:2507.09019v1 [cs.LG] 11 Jul 2025

On Evaluating Performance of LLM Inference Systems

Amey Agrawal[1] Nitin Kedia[2] Anmol Agarwal[1] Jayashree Mohan[2]
Nipun Kwatra[2] Souvik Kundu[3] Ramachandran Ramjee[2] Alexey Tumanov[1]
[1]Georgia Institute of Technology [2]Microsoft Research [3]Intel Labs

Abstract

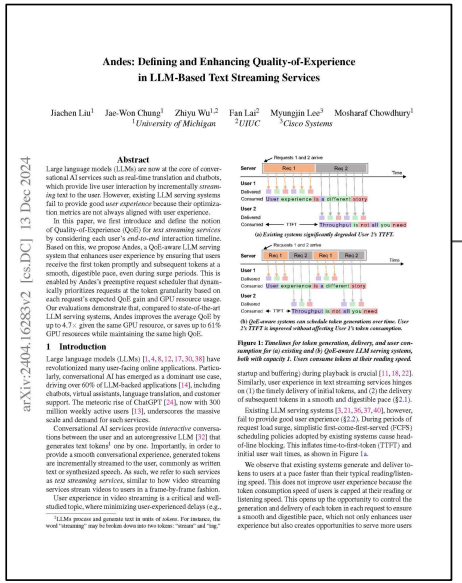
The rapid evolution of Large Language Model (LLM) inference systems has yielded significant performance and efficiency improvements. However, our systematic analysis reveals that current evaluation methodologies frequently exhibit fundamental flaws, often manifesting as common evaluation anti-patterns that obscure true performance characteristics and impede scientific progress. Through a comprehensive examination of recent systems, we identify recurring anti-patterns across three key dimensions: *Baseline Fairness*, *Evaluation Setup*, and *Metric Design*. These anti-patterns are uniquely problematic for LLM inference due to its dual-phase nature combining distinct prefill and decode operations, its handling of highly heterogeneous workloads, and its strict temporal requirements for interactive use. We demonstrate how common anti-patterns—such as inadequate baseline comparisons that conflate engineering effort with algorithmic novelty, workload selections that fail to represent production scenarios, and metric normalizations that hide substantial performance variability like generation stalls—lead to misleading conclusions. To address these challenges, we provide a comprehensive checklist derived from our analysis, establishing a framework for recognizing and avoiding these anti-patterns in favor of robust LLM inference evaluation. To demonstrate the practical application of our framework, we present a case study analyzing speculative decoding, a technique whose benefits are frequently misinterpreted when evaluated using approaches characteristic of these anti-patterns. Our work establishes a rigorous foundation for evaluation methodology, enabling meaningful comparisons, ensuring reproducible results, and ultimately accelerating genuine progress in LLM inference systems by moving beyond common anti-patterns to align evaluation with real-world requirements.

1 Introduction

The past two years have witnessed remarkable progress in Large Language Model (LLM) inference systems. The community has dramatically improved efficiency, achieving orders-of-magnitude higher throughput and lower latency through innovations like continuous batching, paged attention, and speculative decoding Yu et al. (2022); Kwon et al. (2023); Agrawal et al. (2024b); Zhong et al. (2024); Patel et al. (2024). Open-source frameworks and commercial services have scaled these systems to millions, democratizing access to powerful generative AI.

This rapid technological progress, however, has outpaced our evaluation methodologies. Our systematic analysis of recent systems reveals a concerning divergence: while the systems themselves have evolved rapidly, operating at unprecedented scales, our methods for evaluating them remain largely static. This stagnation leads to inconsistencies that obscure genuine performance characteristics and impede scientific advancement. Through a comprehensive examination of influential works from top-tier peer-reviewed venues, we identify some consistent anti-patterns in the evaluation of LLM inference systems.

1



arXiv:2404.16283v2 [cs.DC] 13 Dec 2024

Andes: Defining and Enhancing Quality-of-Experience in LLM-Based Text Streaming Services

Jiachen Liu[1] Jae-Won Chung[1] Zhiyu Wu[1,2] Fan Lai[2] Myungjin Lee[3] Mosharaf Chowdhury[1]
[1]University of Michigan [2]UIUC [3]Cisco Systems

Abstract

Large language models (LLMs) are now at the core of conversational AI services such as real-time translation and chatbots, which provide live user interaction by incrementally streaming text to the user. However, existing LLM serving systems fail to provide good *user experience* because their optimization metrics are not always aligned with user experience.

In this paper, we first introduce and define the notion of Quality-of-Experience (QoE) for *text streaming services* by considering each user's *end-to-end* interaction timeline. Based on this, we propose Andes, a QoE-aware LLM serving system that enhances user experience by ensuring that users receive the first token promptly and subsequent tokens at a smooth, digestible pace, even during surge periods. This is enabled by Andes's preemptive request scheduler that dynamically prioritizes requests at the token granularity based on each request's expected QoE gain and GPU resource usage. Our evaluations demonstrate that, compared to state-of-the-art LLM serving systems, Andes improves the average QoE by up to 4.7× given the same GPU resource, or saves up to 61% GPU resources while maintaining the same high QoE.

1 Introduction

Large language models (LLMs) [1, 4, 8, 12, 17, 30, 38] have revolutionized many user-facing online applications. Particularly, conversational AI has emerged as a dominant use case, driving over 60% of LLM-backed applications [14], including chatbots, virtual assistants, language translation, and customer support. The meteoric rise of ChatGPT [24], now with 300 million weekly active users [13], underscores the massive scale and demand for such services.

Conversational AI services provide *interactive* conversations between the user and an autoregressive LLM [32] that generates text tokens[1] one by one. Importantly, in order to provide a smooth conversational experience, generated tokens are incrementally streamed to the user, commonly as written text or synthesized speech. As such, we refer to such services as *text streaming services*, similar to how video streaming services stream videos to users in a frame-by-frame fashion.

User experience in video streaming is a critical and well-studied topic, where minimizing user-experienced delays (e.g., startup and buffering) during playback is crucial [11, 18, 22]. Similarly, user experience in text streaming services hinges on (1) the timely delivery of initial tokens, and (2) the delivery of subsequent tokens in a smooth and digestible pace (§2.1).

Existing LLM serving systems [3, 21, 36, 37, 40], however, fail to provide good user experience (§2.2). During periods of request load surge, simplistic first-come-first-served (FCFS) scheduling policies adopted by existing systems cause head-of-line blocking. This inflates time-to-first-token (TTFT) and initial user wait times, as shown in Figure 1a.

We observe that existing systems generate and deliver tokens to users at a pace faster than their typical reading/listening speed. This does not improve user experience because the token consumption speed of users is capped at their reading or listening speed. This opens up the opportunity to control the generation and delivery of each token in each request to ensure a smooth and digestible pace, which not only enhances user experience but also creates opportunities to serve more users

[1]LLMs process and generate text in units of *tokens*. For instance, the word "streaming" may be broken down into two tokens: "stream" and "ing."

(b) *QoE-aware systems can schedule token generations over time. User 2's TTFT is improved without affecting User 1's token consumption.*

Figure 1: *Timelines for token generation, delivery, and user consumption for (a) existing and (b) QoE-aware LLM serving systems, both with capacity 1. Users consume tokens at their reading speed.*

Common issues in existing performance evaluation

Inference scheduling optimization by modeling user experience

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

On Evaluating Performance of LLM Inference Systems

Amey Agrawal[1], Nitin Kedia[2], Anmol Agarwal[1], Jayashree Mohan[2], Nipun Kwatra[2], Souvik Kundu[3], Ramachandran Ramjee[2], Alexey Tumanov[1]

[1]Georgia Institute of Technology [2]Microsoft Research [3]Intel Labs

Motivation

- LLM inference systems have yielded significant performance improvements
- Evaluation methods remain static, frequently exhibit flaws

→ Obscure genuine performance characteristics, impede scientific advancements

Challenges in Evaluating LLM Inference Serving Systems

- Dual-Phase Nature
  - Memory-bound Decode / Compute-bound Prefill
  - → Complex performance dynamics, high variability in resource utilization
- Heterogeneous Application Requirements
  - Prompt length
  - Performance metric prioritization
- Fast-Changing Domain & Architectures

Trace	# Prefill tokens			# Decode tokens		
	Median	IQR	P99	Median	IQR	P99
Azure Code 2024	1928	2393	7685	8	15	276
Azure Conv 2024	928	1811	6683	41	94	694
Mooncake	6345	4243	61616	30	343	898

Table 1: Details of public production traces from Stojkovic et al. (2024);

Anti-Patterns

Baseline Fairness	1. Conflating Implementation and Algorithm 2. Neglecting parameter tuning
Evaluation Setup	3. Evaluating with Outdated or Irrelevant Models 4. Using Non-Representative Workloads 5. Ignoring Practical Latency Bounds
Metric Design and Interpretation	6. Selecting Misleading or Opaque Metrics 7. Reporting Only Summary Statistics (Ignoring Distribution) 8. Obscuring Performance with Normalization

Anti-Patterns (+ Evaluation Checklist)

Baseline Fairness	1. Conflating Implementation and Algorithm 2. Neglecting parameter tuning
Evaluation Setup	3. Evaluating with Outdated or Irrelevant Models 4. Using Non-Representative Workloads 5. Ignoring Practical Latency Bounds
Metric Design and Interpretation	6. Selecting Misleading or Opaque Metrics 7. Reporting Only Summary Statistics (Ignoring Distribution) 8. Obscuring Performance with Normalization

Anti-Pattern 1: Conflating Implementation and Algorithm

- Algorithmic Gains vs. Engineering Improvements
  - Scheduling overheads
  - Communication protocols
  - Low-level optimizations
- ❌ Example: DistServe
  - Evaluated on three different codebases (vLLM, DeepSpeed-MII, DistServe)
  - No microbenchmarks to isolate implementation differences from algorithmic benefits

Implementation Fairness:
- Are baselines implemented comparably?
- If not, are microbenchmarks or ablations used to isolate algorithmic gains from system implementation differences?

Anti-Pattern 2: Neglecting Parameter Tuning

- Comparing against baseline using unoptimized parameters
  - Batch size
  - Chunk size
  - Scheduling policies
- ❌ Example: LoongServe
  - Comparing against chunked prefill baselines (DeepSpeed-MII, LightLLM)
  - No report on chunk size for DeepSpeed-MII
  - LightLLM chunk size uses formula from Sarathi despite different operating context

Parameter Tuning:
- Are all performance-critical configuration parameters documented for both the proposed system and baselines?
- Were baseline parameters tuned appropriately for the evaluation setup?

Anti-Pattern 3: Evaluating with Outdated/Irrelevant Models

- Techniques showing benefits on one architecture may not generalize to others
  - Classic choice: MHA (Multi-Head Attention)
  - Newer: GQA (Grouped Query Attention) requires 4-8x less KV cache memory
- ✅ Example: Sarathi-Serve
  - Evaluation covers diverse model sizes (7B, 34B, 70B, 180B)
  - Included MHA + GQA, representing the state-of-the-art model architectures

Model Selection:
- Do the evaluated models reflect current state-of-the-art?
- Are key techniques evaluated across different model architectures where their impact might significantly vary?

Anti-Pattern 4: Using Non-Representative Workloads

- LLMs serve diverse applications with distinct workload characteristics
  - E.g. code completion, chat, summarization
- Evaluating systems on single-source datasets (often with limited input length) → not representative to all workloads
- ❌ Example: vLLM
  - Evaluation focuses on short sequences only
  - With ShareGPT (mean prefill=161, decode=331) and Alpaca (mean prefill=19, decode=58) datasets

Workload Diversity:
- Is the system evaluated beyond a single dataset or synthetic traces?
- Are diverse applications (chat, code, summarization) with their characteristic distributions of prompt/output lengths and interaction models evaluated?

Anti-Pattern 5: Ignoring Practical Latency Bounds

- Presenting improvements without context in practical latency requirements
- Example: ❌ LoongServe
  - Proposes optimizations for long contexts (up to 500k tokens)
  - Reports normalized latencies per input token (mean of prefill time / input lengths)
  - Numbers are small (0-1s), but translates to absolute TTFTs in the range of thousands of s → impractical

Practical Latency Bounds:
- Are reported improvements achieved within latency bounds (e.g., TTFT, TBT SLOs) suitable for the target application(s)?
- Is the connection between reported metrics and absolute user-experienced latency clear?

Anti-Pattern 6: Selecting Misleading or Opaque Metrics

- Heterogeneity in inference workloads lead to fragmentations in evaluation
  - Each request can vary in its characteristics (e.g. prompt/output length, latency sensitivity)
  - Challenging to use traditional metrics, different papers adopting various metrics
    → **Difficult to compare systems fairly** + metrics may **not align well with user experience**
- ❌ Example: dLoRA
  - Reports average latency (sum of each request's end-to-end latency / total #output tokens)
  - Does not correspond to actual latency experienced by any user

Metric Selection:
- Are the chosen metrics broadly used and understood?
- For a novel metric, is its relationship to user-perceived performance explained and justified?

Anti-Pattern 7: Reporting Only Summary Statistics

- Focusing solely on summary statistics (e.g. median latency)
  → …while ignoring the full performance distribution
- ❌ Example: vLLM, Orca
  - Primarily reported median latency
  - Subsequent work revealed these systems suffered from high tail latency

Performance Distribution:

- Does the evaluation present the full latency distribution (e.g., CDFs) or at least key percentiles (P50, P90, P99), rather than just a single summary statistic like average or median?
- Does the analysis illuminate performance trade-offs across different points in the distribution?

Anti-Pattern 8: Obscuring Performance with Normalization

- Normalization may obscure user-facing issues in evaluation
- Example:
  - Two requests generating 10 and 1000 tokens respectively
  - Both experience a 10s scheduling delay, with 20ms per-token generation
  - Normalized latency is dramatically different (1.02 s/token vs. 0.03 s/token)
    → falsely suggesting a vast difference in user experience

No Obscuring Metric Normalization:

- When using normalized metrics (Normalized Latency, TPOT), are raw performance characteristics also examined?
- Are critical temporal aspects like scheduling delays (Ts) and token generation stalls (TBT variance) analyzed independently?
- Does the evaluation consider how normalization might obscure user-facing performance issues evident in raw measurements?

	Orca	*vLLM*	*Sarathi*	*DistServe*	*dLoRA*	*S-LoRA*	*L2R*	*MuxServe*	*SpotServe*	*LoongServe*	*NanoFlow*	*SplitWise*
Baseline Fairness												
Implementation Fairness	✓	✓	✓	×	✓	✓	✓	✓	✓	×	✓	✓
Parameter Tuning	✓	×	✓	×	✓	✓	✓	✓	✓	×	✓	✓
Evaluation Setup												
Model Selection	✓	×	✓	✓	✓	×	✓	✓	×	✓	✓	✓
Workload Diversity	×	×	✓	×	×	×	×	×	×	×	✓	✓
Practical Latency	✓	✓	×	✓	×	×	×	×	×	×	×	✓
Metrics Design												
Metric Selection	✓	✓	✓	✓	×	✓	✓	×	✓	✓	✓	✓
Performance Distribution	×	×	×	✓	×	×	✓	×	✓	×	✓	✓
No Obscuring Normalization	×	×	✓	×	×	✓	×	×	✓	×	×	×

Case Study: Speculative Decoding

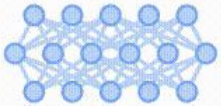
Traditional LLM inference generates one token per full forward pass. Too Slow!

Solution (high-level idea):

- **Smaller draft model**: propose multiple future tokens
- Primary model: in parallel, verify their probability (and potentially override) in one forward pass

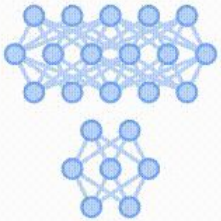
Case Study: Speculative Decoding

WITHOUT SPECULATIVE DECODING



My favorite thing about fall

WITH SPECULATIVE DECODING



My favorite thing about fall

Case Study: Speculative Decoding

This results in a spread of decode latencies…

- Successful verifications

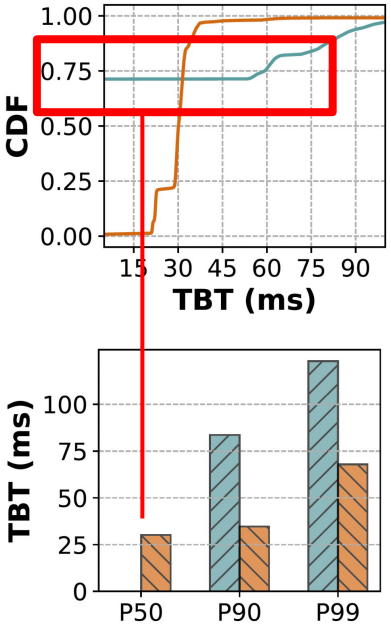
  → Yield bursts of tokens arriving almost instantly

- Verification failures

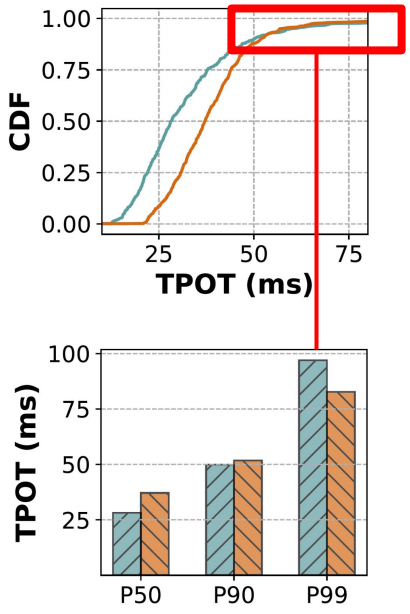
  → Delays, as only a single token is produced after incurring costs for both draft generation + verification

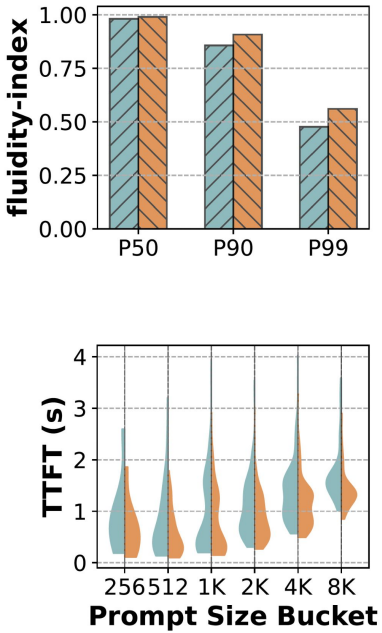
Case Study: Speculative Decoding

(AP 7) TBT becomes misleading



(AP 5 & 8) TTFT is negatively impacted





(AP 7 & 8) Conflicting trends with TPOT

Conclusion

Contributions:

- **Analysis of common evaluation anti-patterns**
- **Evaluation checklist** (guidance for meaningful evaluation → genuine user experience enhancements)
- **Case study** on speculative decoding (revealed performance trade-offs that traditional metrics obscure)

Limitations & Future Work

- Generalizability
- Practicality
  - Subjectivity: “fair tuning”, “representative workloads”
  - How to reconcile with other objectives (e.g., cost)

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

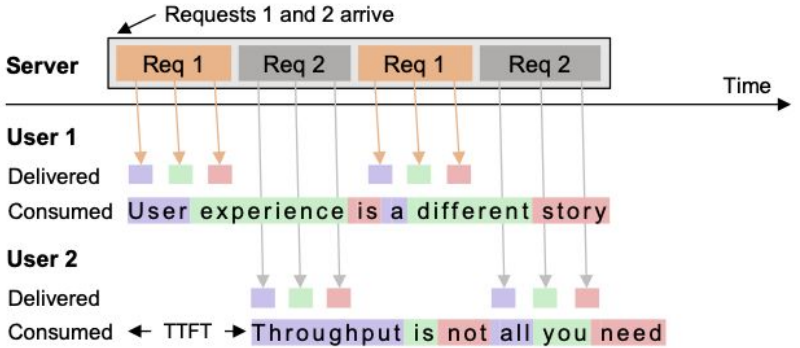
Andes: Defining and Enhancing Quality-of-Experience in LLM-Based Text Streaming Services

Jiachen Liu[1], Jae-Won Chung[1], Zhiyu Wu[1, 2], Fan Lai[2], Myungjin Lee[3], Mosharaf Chowdhury[1]

[1]University of Michigan [2]UIUC [3]Cisco Systems

Background: Text Streaming Services

- Audio & Textual tokens streamed to user
  - Initial Token Delivery (**TTFT**)
  - Subsequent Token Delivery Speed (**TDS**)
- Head-of-line blocking
  - Server centered metrics



(b) ***QoE-aware systems can schedule token generations over time. User 2's TTFT is improved without affecting User 1's token consumption.***

Motivation: User Experience

- Misallocation of resources over time in existing systems
  - **Inconsistent TDS** and **poor TTFT** under load
- Motivates preemptive scheduling approach
  - Overhead

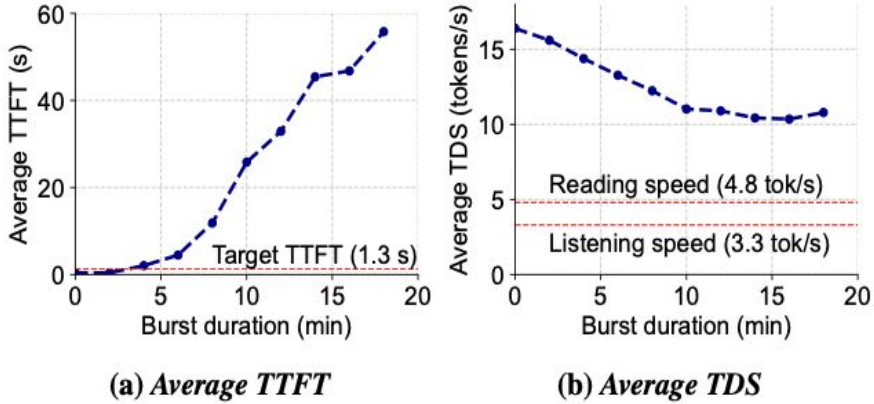
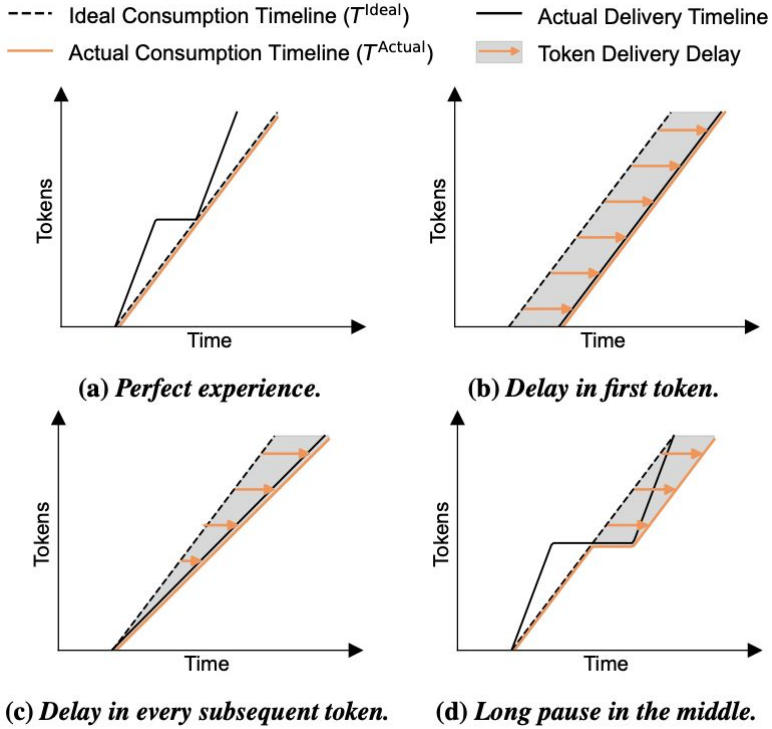


Figure 4: ***vLLM's average TTFT and TDS while varying the duration of the load surge in our 20-minute trace.***

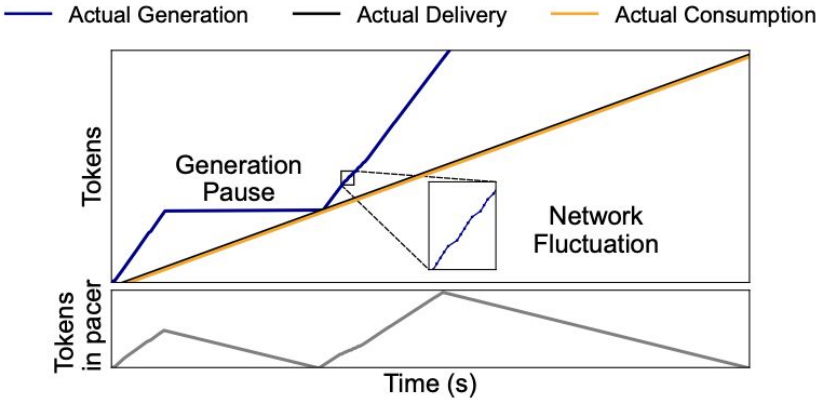
Andes: Quality-of-Experience (QoE)

- Intuition
  - Should capture entire token-consumption timeline (without averages)
  - Should be unaffected by TDS that is faster than user's consumption speed
  - Degradation from earlier delays should be diluted
- Parameters
  - TTFT Target
  - User Reading/Listening Speed



Andes: Architecture

- Goal: Maximize average QoE across requests
- **Request Tracker**
- **Token-Level Request Scheduler**
- Executor + KV Cache
- **Token Pacer**



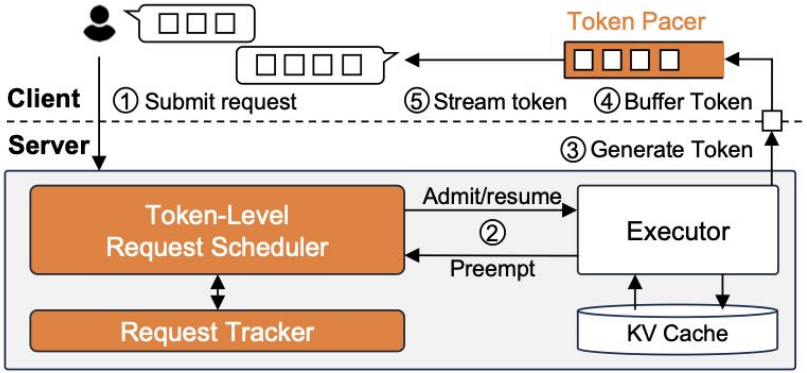


Figure 6: ***High-level architecture of Andes. Components designed by Andes are colored in orange.***

QoE-Aware Token-Level Scheduler

- Preemptively pauses/resumes requests at token granularity
- Requests are served in order of QoE gain
  - ($Q_{\text{serve},i} - Q_{\text{wait},i}$)
  - Gain estimated over upcoming Δt
- O(*MN^3*) to solve for B

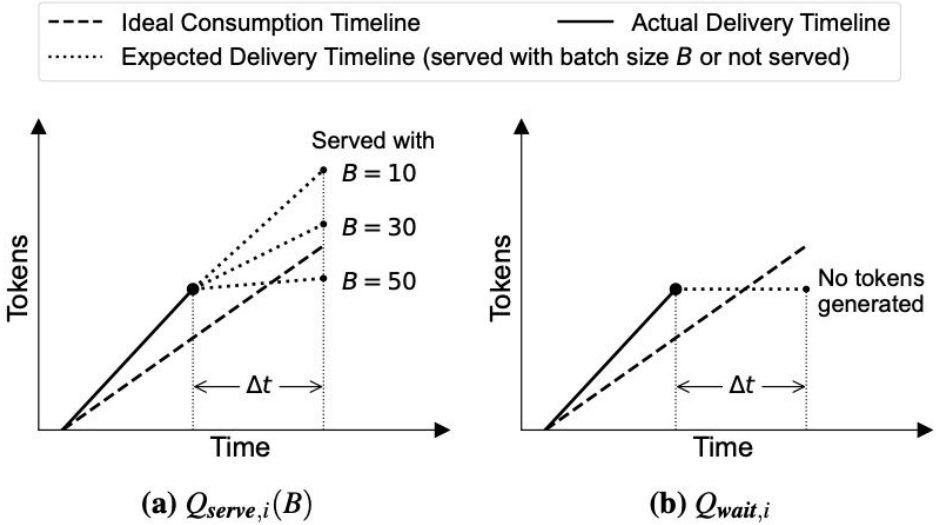


Figure 7: ***Visualization of*** $Q_{serve,i}(B)$ ***and*** $Q_{wait,i}$.

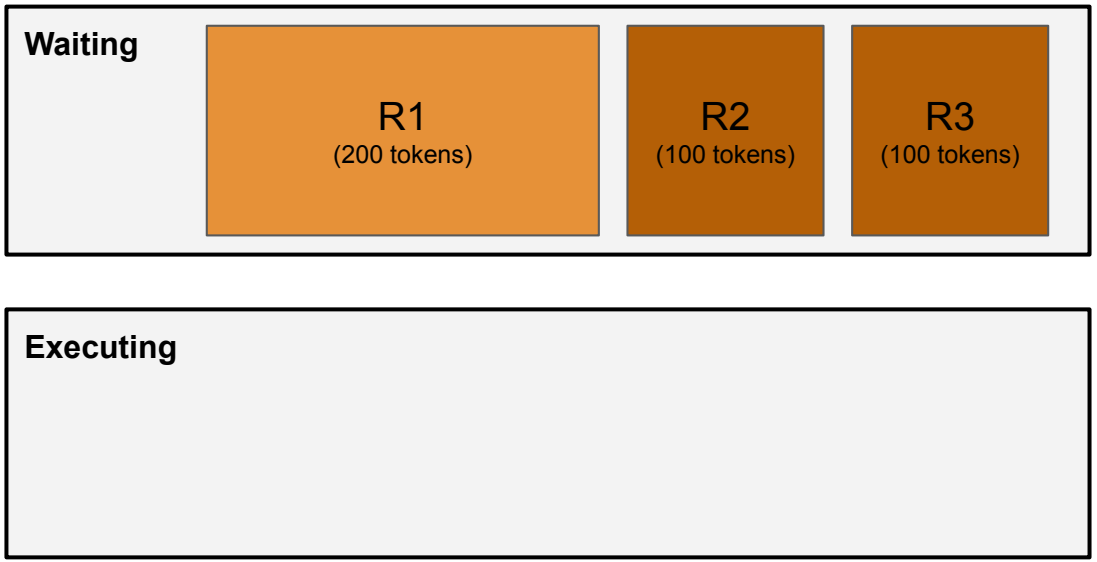
Priority-Based Scheduler

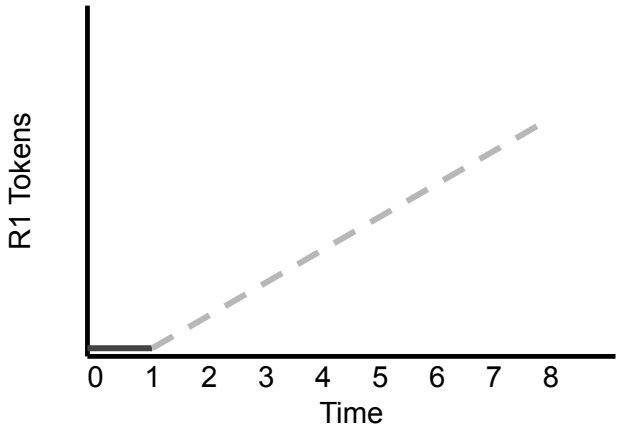
- Priority based greedy heuristic
- O(*NlogN*)
- Implicitly prevents starvation as older requests have longer contexts
- Similar QoE to DP solution but 20x faster

$$\frac{Q_{\text{serve},i}(B) - Q_{\text{wait},i}}{l_i}$$

Scheduler Walkthrough

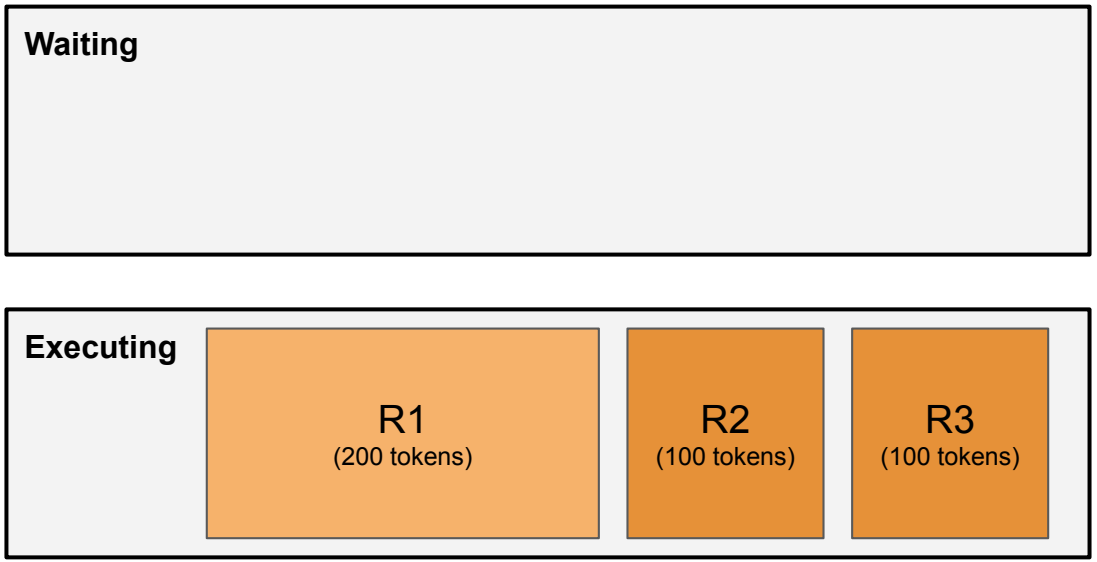
T = 0

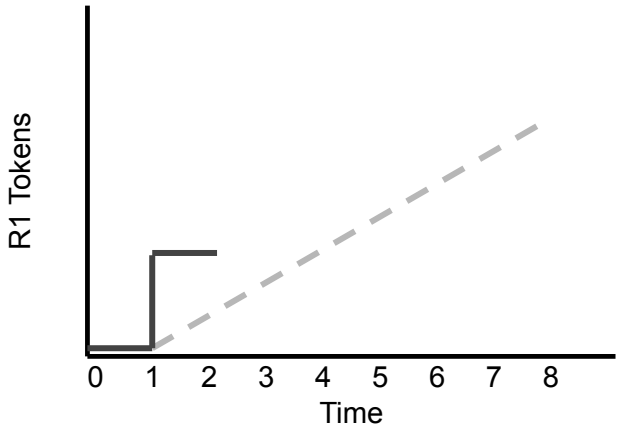




Scheduler Walkthrough

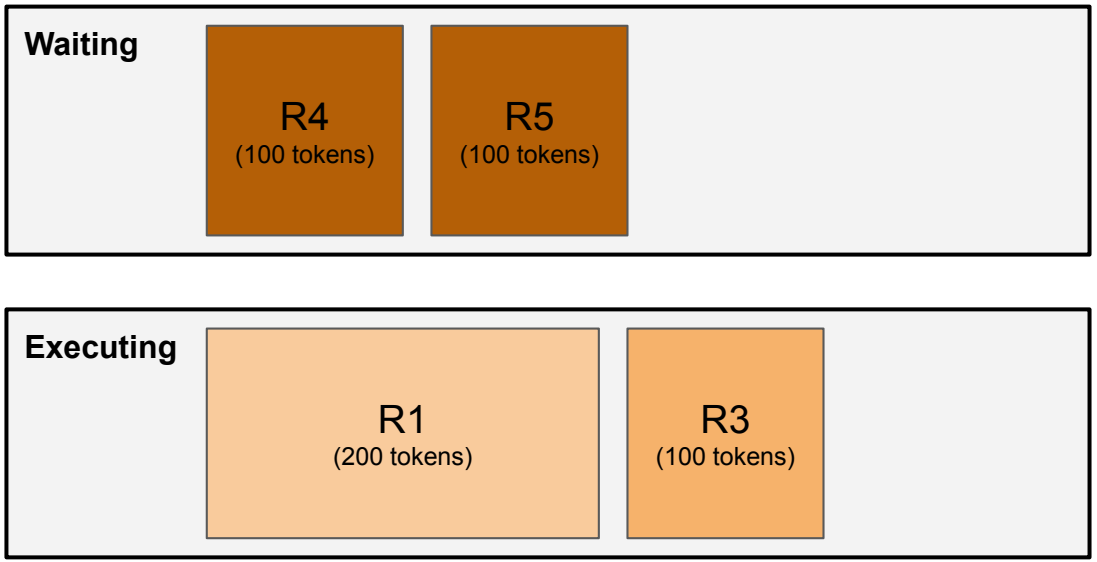
T = 1: Requests Start Executing

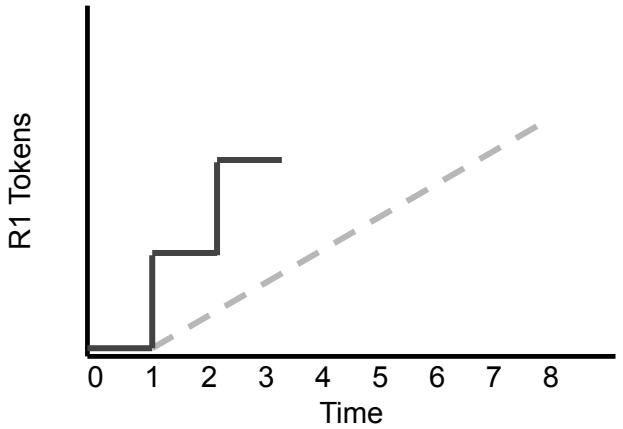




Scheduler Walkthrough

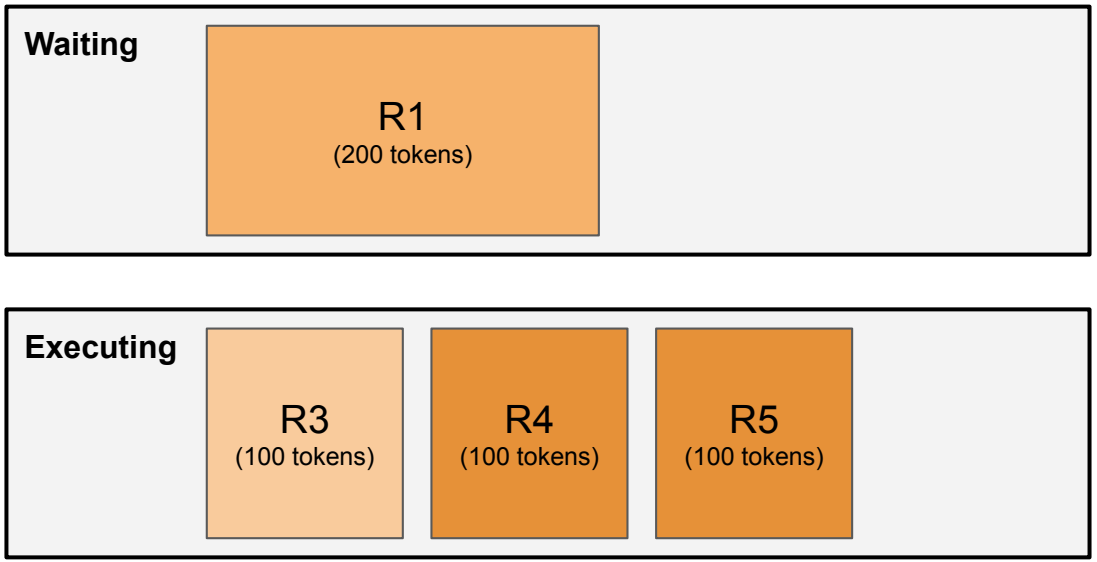
T = 2: R2 Completes and R4, R5 introduced

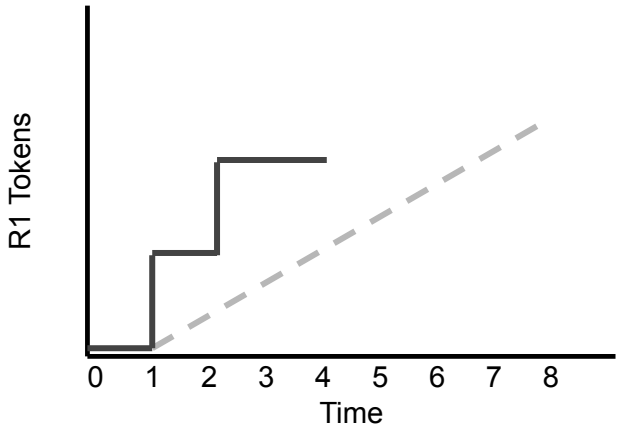




Scheduler Walkthrough

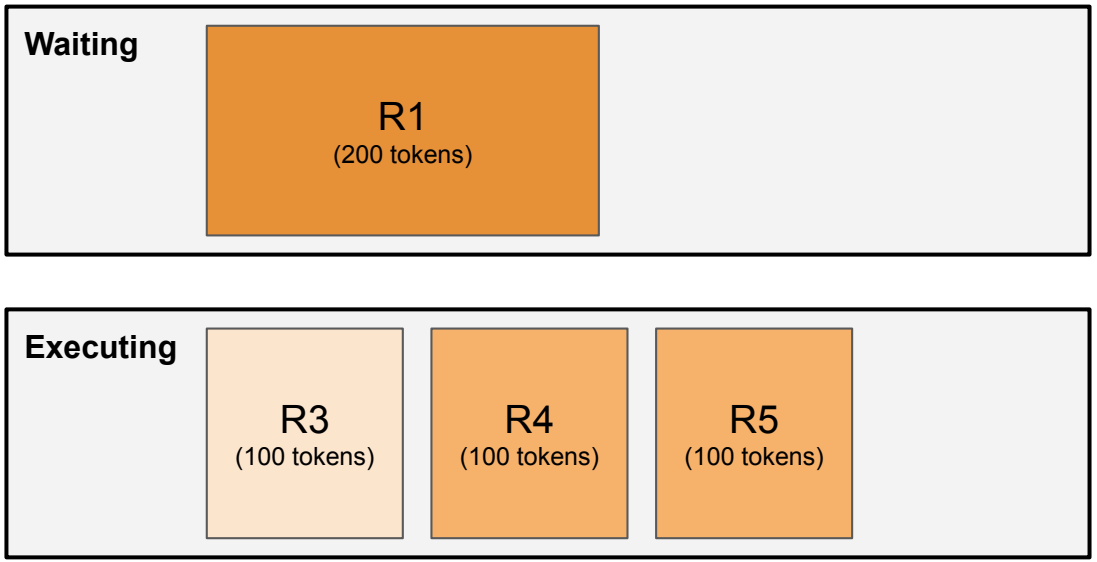
T = 3: R1 Preempted

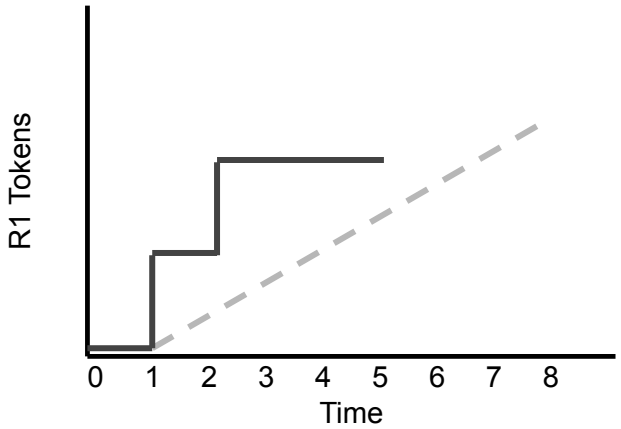




Scheduler Walkthrough

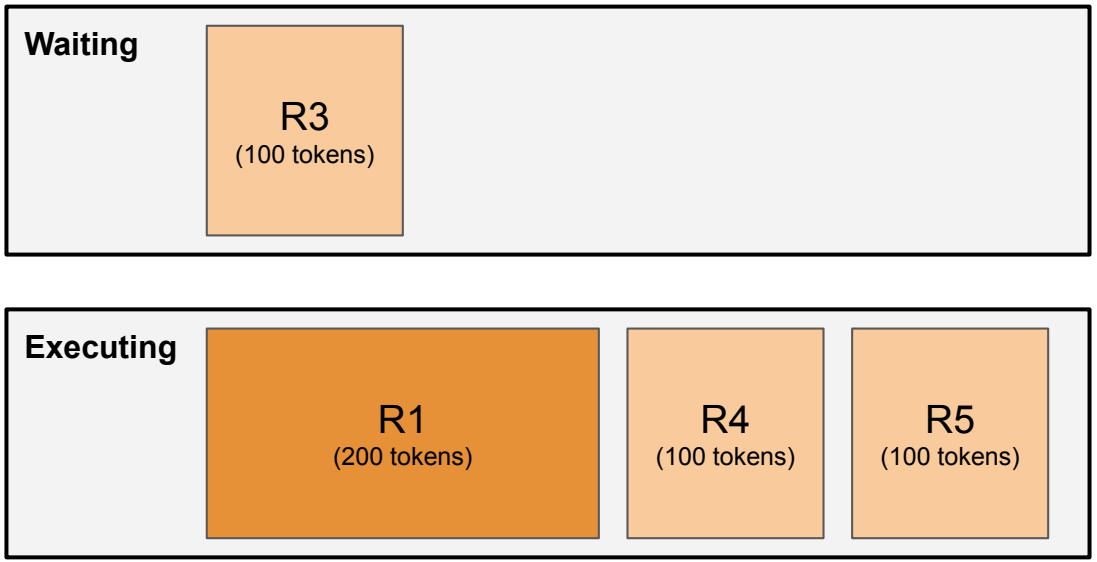
T = 4: R1 Close to threshold

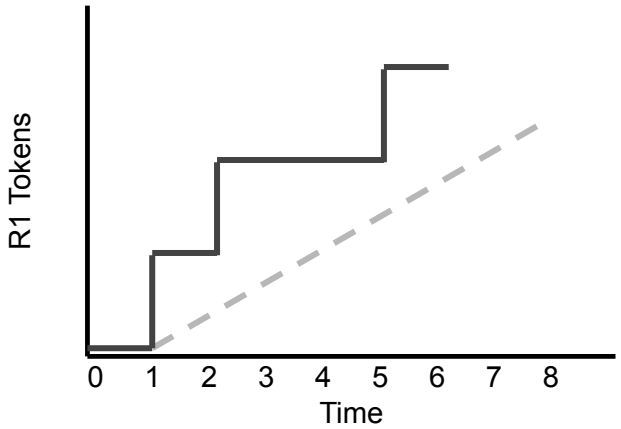




Scheduler Walkthrough

T = 4: R3 Preempted





Incorporating Preemption Overhead

- Preemption Tradeoff
  - Optimize scheduler choices to account for preemption overhead (swapping/recomputation)
- Refiner selectively chooses requests to not preempt

Incorporating Preemption Overhead

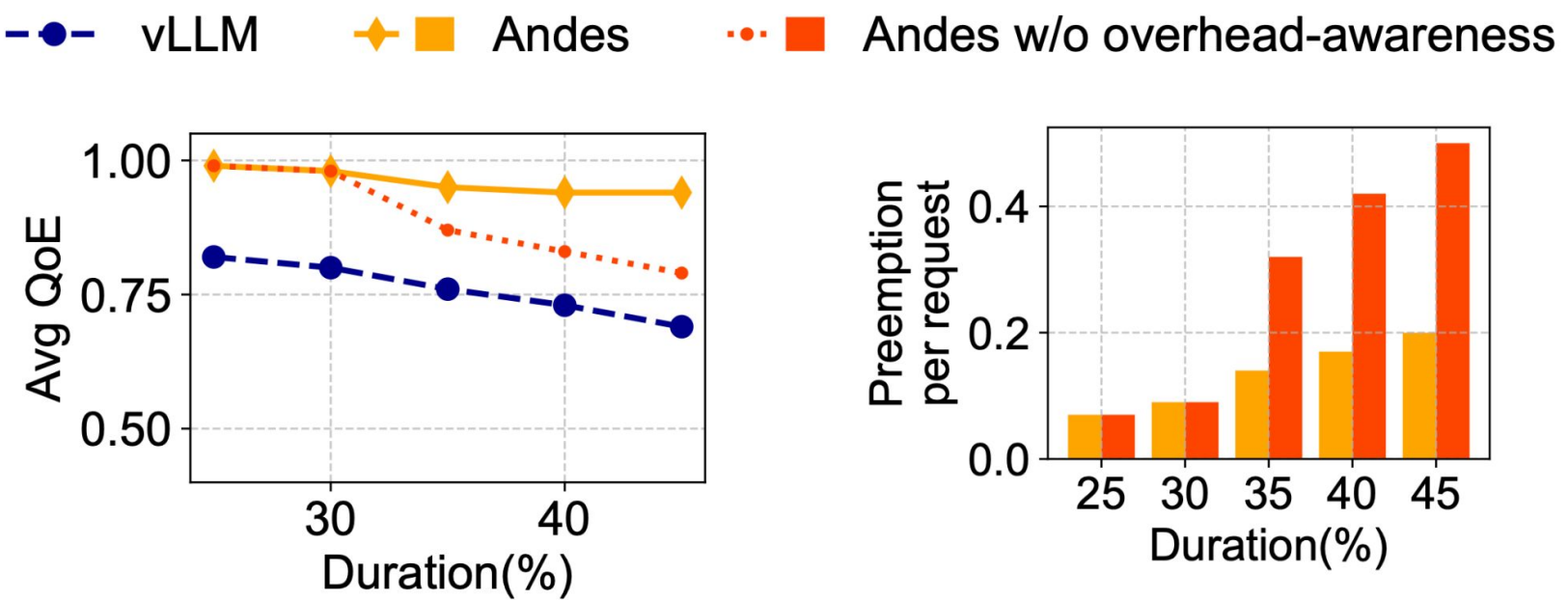


Figure 17: ***Overhead-aware refiner is critical to optimize QoE.***

Results: Experimental Setup

- Evaluated on variety of models & architectures (Dense, MoE, Multi-Head, Grouped-Query)
- Metrics: Average QoE, GPU Resource Savings(%)
- Datasets: Real world traces from BurstGPT and synthetic bursty traces

Model	Architecture	Memory	Hardware
Phi-3-mini 3.8B [1]	Dense, MHA	7 GB	A100×4
Command R 32B [10]	Dense, GQA	61 GB	A100×8
Phi-3.5-MoE 16×3.8B [1]	MoE, GQA	80 GB	A100×8
Llama 3.1 70B [12]	Dense, GQA	132 GB	A100×8

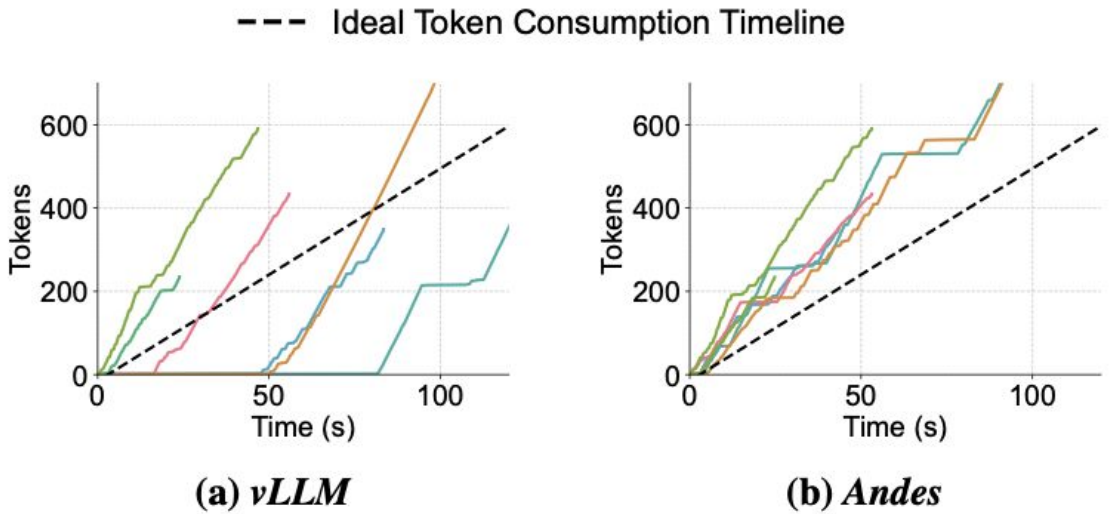
Table 1: ***Models and hardware configurations.***

Dataset	Input Length		Output Length	
	Mean	Std.	Mean	Std.
Multi-Round ShareGPT [31]	3171	7943	385	300
ArXiv Summarization [9]	17855	11401	605	153
Coding Challenges [16]	675	1552	5423	21293

Table 2: ***Request dataset statistics.***

Results: Real-World Traces

- Andes compared to vLLM w/ FCFS
- Andes can maintain high QoE with less GPUs or serve more with same GPUs



Results: Real-World Traces

- Andes can reduce peak waiting queue length by 85%

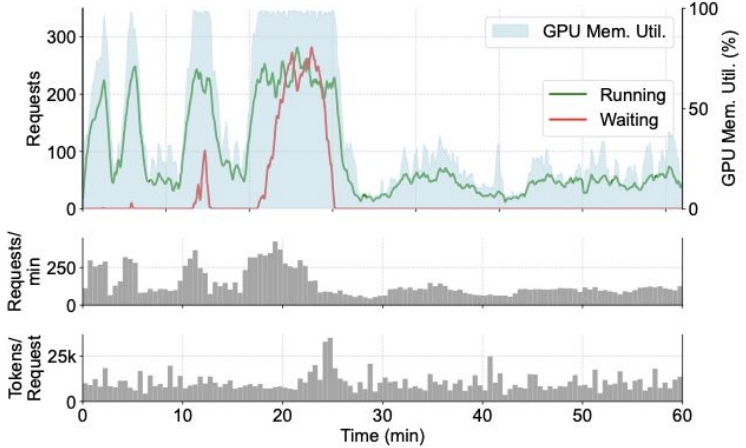


Figure 3: *vLLM serving requests from BurstGPT.*

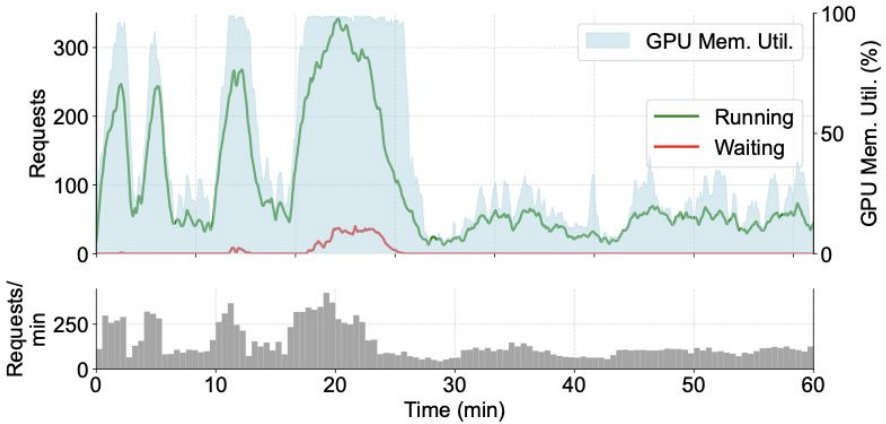


Figure 13: *Andes serving requests from BurstGPT.*

Strength & Weaknesses

- Strengths:
  - Intuitive QoE metric
  - Accounts for preempting overhead
  - Approximate method is close to optimal
- Weaknesses:
  - Study limited to BurstGPT and synthetic data, no live experimentation
  - Assumes users have fixed token reading/listening speed
  - Only works for Audio and Text outputs

Related Works & Future Directions

- SplitWise, DistServe, Sarathi-Serve, & LoongServe keep the FCFS scheduler
- VTC develops a non-preemptive fairness-based scheduler
- Video streaming
  - Network vs Memory & Compute constraints
  - No parallels to metrics like buffering time, average bitrate
- Future
  - Explore dynamic token consumption timelines
  - Integration with multi-server clusters

Summary

	On Evaluating Performance of LLM Inference Systems	**Andes: Defining and Enhancing Quality-of-Experience in LLM-Based Text Streaming Services**
Goal	Identify evaluation pitfalls and propose a robust benchmarking framework	Define & optimize user-centric QoE for streaming LLM services
Core Contribution	Taxonomy of evaluation anti-patterns + checklist for proper benchmarking	New QoE metric + preemptive token-level LLM scheduler (w/ overhead-aware refiner) + client token pacer
Metrics	TTFT, TBT, distributions	TTFT, QoE curve vs ideal consumption timeline
Key Insights	Throughput-only metrics mislead; proper evaluation demands latency distribution, realistic workloads, and fairness	Users can only read so fast; smooth streaming & fast first token matter more than raw token/sec